



دانشگاه فردوسی مشهد
گروه مهندسی کامپیوتر

گزارش پروژه درس
مبانی هوش محاسباتی

Persian Sentiment Analysis with Ensemble Learning

تحلیل احساسات نظرات فارسی با یادگیری ترکیبی

نام و نام خانوادگی	شماره دانشجویی
امیرحسین ابوالفضلی	4022262035
آرمان بیجاری	4021262131

استاد: دکتر فضل ارثی

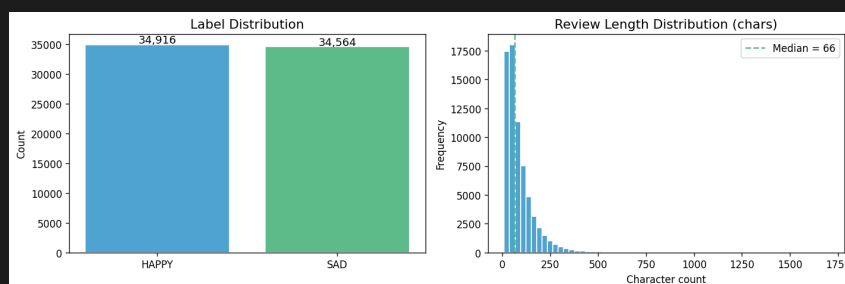
نیمسال دوم 1404-1405

فهرست مطالب

۳	۱ معرفی داده‌ها
۳	۲ فاز اول: پیش‌پردازش متن
۳	۱.۲ چرا فارسی سخت‌تره؟
۳	۲.۲ مراحل پیش‌پردازش
۴	۳.۲ نمونه‌هایی از قبل و بعد
۴	۴.۲ تقسیم داده
۴	۳ فاز دوم: Vectorization
۴	۱.۳ CountVectorizer و TF-IDF
۴	۲.۳ Word2Vec
۵	۳.۳ نمایش‌های bonus
۵	۴ فاز سوم: آموزش و ارزیابی مدل‌های پایه
۵	۱.۴ مدل‌ها و تنظیمات
۶	۲.۴ نتایج کامل
۸	۳.۴ تحلیل نتایج
۸	۴.۴ تنظیم ابرپارامتر و ablation پیش‌پردازش
۸	۵.۴ نمونه پیش‌بینی‌های اشتباه (بهترین مدل sweep)
۸	۵ فاز چهارم: انتخاب بهترین Vectorizer
۹	۱.۵ میانگین و اوج عملکرد
۱۰	۲.۵ چرا Hybrid انتخاب شد؟
۱۰	۶ فاز پنجم: Stacking
۱۰	۱.۶ طراحی
۱۰	۲.۶ نتایج
۱۱	۳.۶ تحلیل
۱۲	۴.۶ نمونه‌های خطا و سقف دقت
۱۲	۷ فاز ششم: ParsBERT (bonus)
۱۲	۱.۷ ParsBERT چیه و چرا اصلاً امتحانش کردیم؟
۱۲	۲.۷ پیاده‌سازی و محیط اجرا
۱۳	۳.۷ مقایسه با baseline کلاسیک
۱۴	۴.۷ جمع‌بندی فاز ۶
۱۴	۸ فاز ۷۲ v2 (06_v2): end-to-end Snappfood-BERT
۱۴	۱.۸ چی کار کردیم؟
۱۴	۲.۸ نتایج

۱. معرفی داده‌ها

داده‌ای که توی این پروژه باهاش کار کردیم Snapfood Persian Sentiment Analysis هستش که از Kaggle^۱ برداشتیم. داده 70,000 تا نظر کاربری Snapfood داره که بعد از سفارش غذا نوشتن. هر نظر یه برچسب داره: HAPPY (مثبت) یا SAD (منفی). بعد از فیلتر اولیه 69,480 ردیف مونده (HAPPY: 34,916 / SAD: 34,564) و بعد از پاکسازی 69,479 ردیف (1 نظر خالی حذف شد). از نظر تعداد کلاس متوازنه. یه چیزی که داده رو جالب و در عین حال چالش‌برانگیز می‌کنه اینه که نظرات کاملاً واقعی و عامیانه‌ان. غلط نگارشی، کشیدن حروف، emoji، قاطی فارسی-انگلیسی، همه هست. اینا نویز حساب میشن ولی بخشی‌شون خودش سیگنال احساسه.



شکل ۱: توزیع برچسب‌های مثبت و منفی در داده

پس داده خوبیه، ولی چون عامیانه و غیررسمیه، پیش‌پردازش درست خیلی مهمه. یه چیز دیگه که توی ارزیابی مدل خیلی اثر می‌ذاره اینه که برچسب‌ها همیشه تمیز نیستن. حدود 22% نظرها «ولی» یا «اما» دارن؛ حدود 6% همزمان کلمات مثبت و منفی دارن. برای انسان قابل فهمه، ولی برای مدلی که فقط یه برچسب SAD/HAPPY می‌بینه گیج‌کننده‌ست. پس سقف دقت فقط با مدل بهتر بالا نمیره، بخشی‌ش سقف ذاتی برچسب‌گذاریه.

۲. فاز اول: پیش‌پردازش متن

۱.۲. چرا فارسی سخت‌تره؟

یه مشکل خاص فارسی اینه که بعضی حروف با عربی یونیکد مشترک ندارن ولی عین هم به نظر میان. «ی» فارسی U+06CC و «ی» عربی U+064A. از دید مدل دو کلمه جدا میشن مگر اینکه اول نرمال‌سازی کنیم.

۲.۲. مراحل پیش‌پردازش

کد توی src/preprocessing.py هست. برخلاف یه pipeline عمومی، اینجا عمداً سیگنال احساس رو نگه می‌داریم: اول URL حذف میشه (قبل از Hazm^۲ چون نرمالایزر // رو می‌بلعه). بعد emoji‌های احساسی به EMO_NEG/EMO_POS نگاشت میشن، نه اینکه کورکورانه حذف بشن. نرمال‌سازی فارسی، حذف اعداد و علائم، جمع‌کردن حروف تکراری (خووب → خوب)، توکن‌سازی، و مهم‌تر از همه برچسب‌گذاری نفی: بعد نفی‌کننده‌ها مثل «نه» به کلمات بعدی پیشوند NEG_ می‌خورن (ایده Pang & Lee). حذف stopword

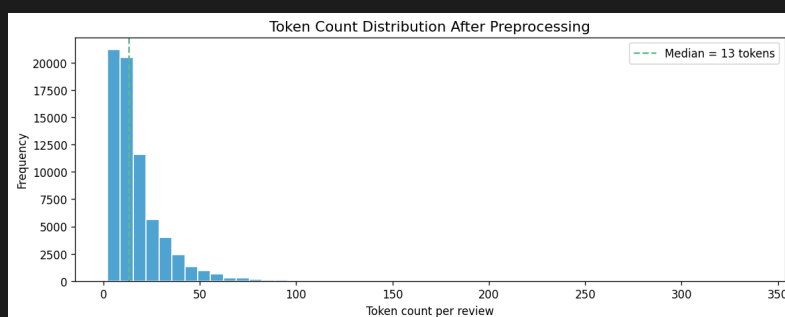
^۱Kaggle: پلتفرم آنلاین برای مسابقات یادگیری ماشین و به اشتراک‌گذاری دیتاست.
^۲Hazm: کتابخانه پایتون برای پردازش زبان طبیعی فارسی.

پیش فرض خاموشه چون لیست Hazm نفی کننده و شدت دهنده داره. آخر سر lemmatization با Hazm.Lemmatizer() توکن های NEG_/EMO_ دست نمی خورن.

۳.۲. نمونه هایی از قبل و بعد

متن اصلی	بعد از پیش پردازش
سلام خیلی خووووووب بود! غذا عالییی بود	سلام خیلی خوب بود غذا عالی بود
بدترین تجربه زندگیم بود. اصلا توصیه نمیکنم	بدترین تجربه زندگیم بود اصلا توصیه کرد به هیچ کس
غذا وقتی رسید سرد بوووووود. افتضاح	غذا وقتی رسید سرد بوود افتضاح

جدول ۱: نمونه های واقعی از نوت بوک فاز اول



شکل ۲: توزیع تعداد توکن (میانگین ۱۷، میانه ۱۳)

۴.۲. تقسیم داده

با stratified split^۳ و random_state=42 آموزش 55,583 (27,650 SAD / 27,933 HAPPY)، آزمون 13,896 (6,913 SAD / 6,983 HAPPY). میانگین طول متن 89.7 کاراکتر (میانه 66).

۳. فاز دوم: Vectorization

سه روش اجباری پروژه رو پیاده کردیم، به علاوه دو نمایش bonus که توی فازهای بعدی مقایسه شدن.

۱.۳ TF-IDF و Count Vectorizer

هر دو sparse با max_features=50,000، ngram_range=(1,2)، min_df=2، TF-IDF با sublinear_tf=True. هر دو ماتریس 50,000 بعدی با پراکندگی 99.9466% ساختن.

۲.۳ Word2Vec

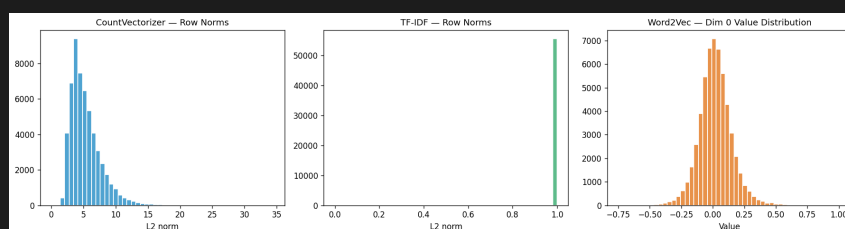
Skip-gram، vector_size=100، آموزش از صفر روی کورپوس. میانگین بردار کلمات برای هر سند. OOV روی train حدود 53.9% - برای نظرات کوتاه و عامیانه طبیعی. stratified split^۳: نسبت کلاس ها توی train و test یکی بمونه.

۳.۳. نمایش‌های bonus

Word2Vec-IDF: میانگین وزن‌دار با IDF (ایده شبیه SIF). Hybrid word+char TF-IDF: اتحاد word و char_wb؛ n-gram: 101,119 بعد با پراکندگی 99.7875%. برای املای غلط و کشیدگی فارسی خیلی کمک می‌کند.

نمایش	بعد	نوع	پراکندگی
CountVectorizer	50,000	sparse	99.9466%
TF-IDF	50,000	sparse	99.9466%
Word2Vec (mean)	100	dense	—
Word2Vec-IDF	100	dense	—
Hybrid	101,119	sparse	99.7875%

جدول ۲: خلاصه نمایش‌های عددی



شکل ۳: توزیع نُرم بردارها

۴. فاز سوم: آموزش و ارزیابی مدل‌های پایه

۱.۴. مدل‌ها و تنظیمات

9 مدل کلاسیک روی پنج vectorizer (سه اجباری + دو bonus): SVM، GaussianNB، ComplementNB، KNN، Random Forest، Decision Tree، Logistic Regression، CalibratedClassifierCV (Linear)، HistGradientBoosting، AdaBoost. ComplementNB فقط روی sparse غیرمنفی؛ GaussianNB روی dense؛ Word2Vec؛ HistGradBoost روی dense. جمعاً 32 آزمایش در sweep اولیه، بعد GridSearchCV برای سه مدل خطی/NB برتر.

مدل	تنظیمات sweep
ComplementNB / GaussianNB	alpha=0.1
SVM (Linear)	CalibratedClassifierCV در C=1.0, max_iter=4000
LogisticRegression	C=1.0, solver=lbfgs
Random Forest	n_estimators=200
HistGradientBoosting	max_iter=400 (فقط dense)
KNN	n_neighbors=7, metric=cosine

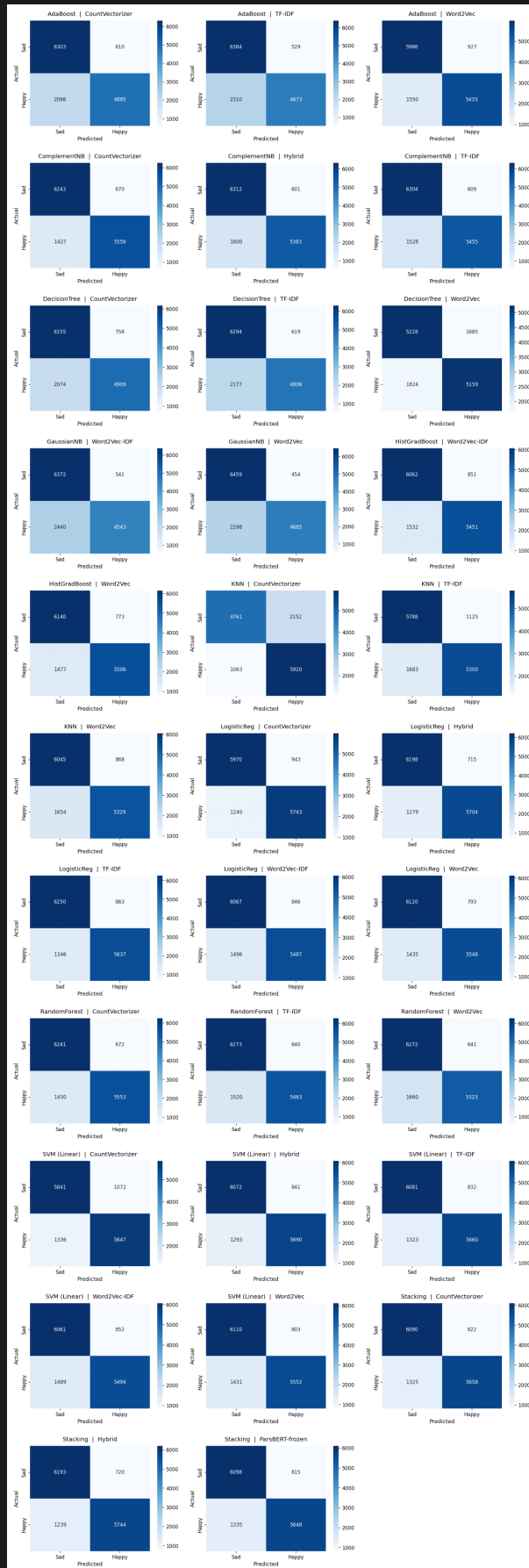
جدول ۳: مدل‌های پایه

۲.۴. نتایج کامل

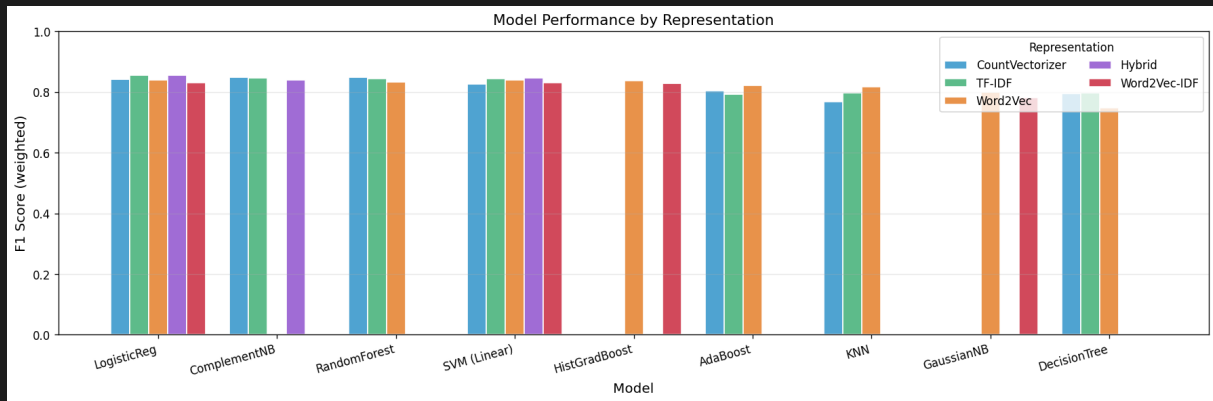
جدول زیر همه 32 ترکیب (شامل سه مدل شده tune در انتها) رو بر اساس F1 مرتب کرده:

F1	فراخوانی	صحت	دقت	Vectorizer	مدل
0.8574	0.8578	0.8620	0.8578	Hybrid	SVM__(Linear)_(tuned)
0.8566	0.8569	0.8602	0.8569	Hybrid	LogisticReg_(tuned)
0.8563	0.8565	0.8589	0.8565	Hybrid	LogisticReg
0.8551	0.8554	0.8590	0.8554	TF-IDF	LogisticReg
0.8501	0.8505	0.8545	0.8505	CountVectorizer	ComplementNB_(tuned)
0.8487	0.8491	0.8534	0.8491	CountVectorizer	ComplementNB
0.8483	0.8487	0.8530	0.8487	CountVectorizer	RandomForest
0.8463	0.8464	0.8480	0.8464	Hybrid	SVM__(Linear)
0.8456	0.8462	0.8525	0.8462	TF-IDF	ComplementNB
0.8448	0.8449	0.8467	0.8449	TF-IDF	SVM__(Linear)
0.8440	0.8446	0.8503	0.8446	TF-IDF	RandomForest
0.8428	0.8429	0.8436	0.8429	CountVectorizer	LogisticReg
0.8408	0.8416	0.8489	0.8416	Hybrid	ComplementNB
0.8394	0.8397	0.8427	0.8397	Word2Vec	LogisticReg
0.8389	0.8392	0.8421	0.8392	Word2Vec	SVM__(Linear)
0.8377	0.8381	0.8417	0.8381	Word2Vec	HistGradBoost
0.8336	0.8344	0.8419	0.8344	Word2Vec	RandomForest
0.8312	0.8315	0.8344	0.8315	Word2Vec-IDF	SVM__(Linear)
0.8311	0.8315	0.8345	0.8315	Word2Vec-IDF	LogisticReg
0.8281	0.8285	0.8318	0.8285	Word2Vec-IDF	HistGradBoost
0.8267	0.8267	0.8272	0.8267	CountVectorizer	SVM__(Linear)
0.8214	0.8217	0.8244	0.8217	Word2Vec	AdaBoost
0.8180	0.8185	0.8227	0.8185	Word2Vec	KNN
0.8030	0.8051	0.8200	0.8051	CountVectorizer	AdaBoost
0.7985	0.8020	0.8251	0.8020	Word2Vec	GaussianNB
0.7976	0.7979	0.7999	0.7979	TF-IDF	KNN
0.7963	0.7988	0.8148	0.7988	TF-IDF	DecisionTree
0.7945	0.7962	0.8074	0.7962	CountVectorizer	DecisionTree
0.7924	0.7957	0.8167	0.7957	TF-IDF	AdaBoost
0.7816	0.7855	0.8088	0.7855	Word2Vec-IDF	GaussianNB
0.7671	0.7686	0.7752	0.7686	CountVectorizer	KNN
0.7475	0.7475	0.7476	0.7475	Word2Vec	DecisionTree

جدول ۴: نتایج کامل 32 آزمایش



شکل ۴: ماتریس درهم‌ریختگی همه مدل‌ها



شکل ۵: مقایسه F1 روی پنج vectorizer

۳.۴. تحلیل نتایج

بهترین ترکیب بعد از GridSearchCV: SVM (Linear) (tuned) با Hybrid روی $F1=0.8574$ و دقت 0.8578. LogisticRegression روی Hybrid و TF-IDF هم نزدیک (0.8563 و 0.8551). Random Forest دیگه برنده نیست؛ روی های sparse پُر بُعد، مدل های خطی با C درست قوی ترن. بدترین ها همچنان DecisionTree روی Word2Vec ($F1=0.7475$) و KNN روی sparse (0.7671) روی CountVectorizer (KNN توی فضای 50k بعدی sparse گم میشه).

۴.۴. تنظیم ابرپارامتر و ablation پیش پردازش

GridSearchCV با fold:5 - ComplementNB $\alpha=1.0$ روی CountVectorizer ($F1=0.8501$)؛ SVM $C=0.05$ روی Hybrid ($F1=0.8574$)؛ LogisticReg $C=0.5$ ($F1=0.8566$). روی زیرنمونه 15k/5k، حذف lemmatization ($F1=0.8572$) از pipeline فعلی (0.8512) جلو بود، ولی pipeline فعلی رو نگه داشتیم چون ablation کوچک بود و negation/emoji-mapping عمداً برای احساس طراحی شده.

۵.۴. نمونه پیش بینی های اشتباه (بهترین مدل sweep)

متن	برچسب	پیش بینی
کیفیت خوب بود ولی نسبت به قیمت حجم غذا خوب بود	SAD	HAPPY
هسته زردآلو خیلی بد بود ولی برگه هلو خوب بود	SAD	HAPPY
سالاد سزار کاهو موند رنگ عوض شد سیب زمینی ساده خوب بود	HAPPY	SAD

جدول ۵: خطاهای LogisticReg + Hybrid – الگوی «ولی»

پس برنده sweep اولیه LogisticReg + Hybrid بود (0.8563)؛ بعد از SVM + Hybrid tune، به 0.8574 رسید.

۵. فاز چهارم: انتخاب بهترین Vectorizer

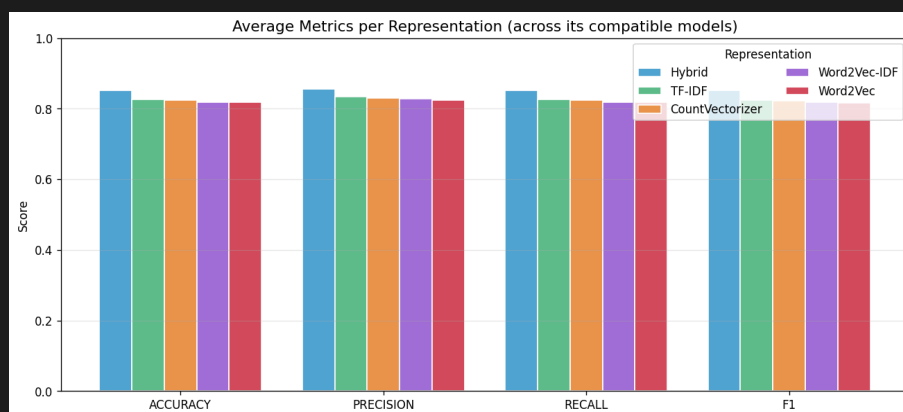
۱.۵. میانگین و اوج عملکرد

Vectorizer	دقت	صحت	فراخوانی	F1
Hybrid	0.8518	0.8556	0.8518	0.8515
TF-IDF	0.8262	0.8343	0.8262	0.8251
CountVectorizer	0.8235	0.8293	0.8235	0.8226
Word2Vec-IDF	0.8193	0.8274	0.8193	0.8180
Word2Vec	0.8176	0.8235	0.8176	0.8169

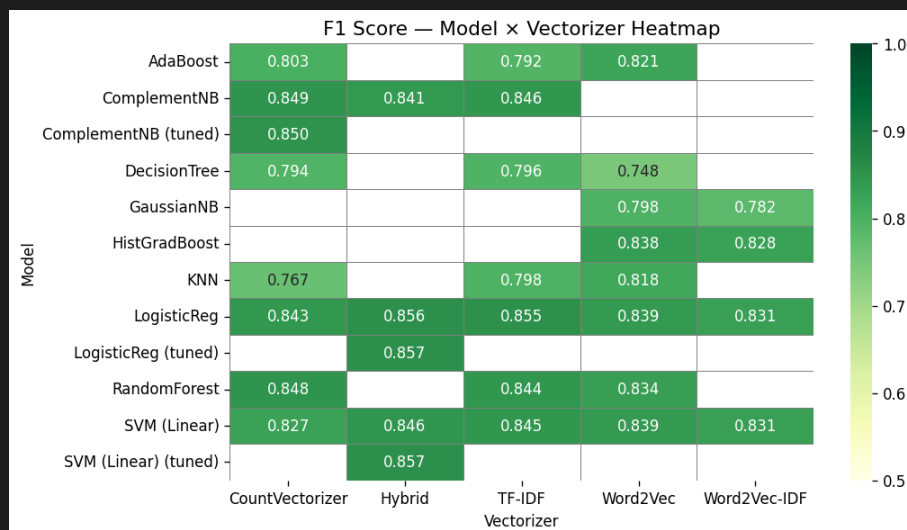
جدول ۶: میانگین F1 روی همه مدل‌های هر vectorizer

Vectorizer	اوج F1
Hybrid	0.8574
TF-IDF	0.8551
CountVectorizer	0.8501
Word2Vec	0.8394
Word2Vec-IDF	0.8312

جدول ۷: بالاترین F1 هر نمایش (بعد از tune)



شکل ۶: میانگین معیارها



شکل ۷: هیتمپ F1

۲.۵. چرا Hybrid انتخاب شد؟

Hybrid هم میانگین (0.8515) هم اوج (0.8574) رو برد. n-gram char املای غلط و پسوند فارسی رو می‌گیره؛ TF-IDF word هم عبارت‌هایی مثل NEG_ رو نگه می‌داره. بین سه روش اجباری، TF-IDF برنده‌ست (0.8551) (اوج)، نه CountVectorizer (0.8501). Hybrid در عمل نسخه تقویت‌شده TF-IDF با union char هست. پس برای فاز Stacking، Hybrid ذخیره شد (best_vectorizer.json).

۶. فاز پنجم: Stacking

۱.۶. طراحی

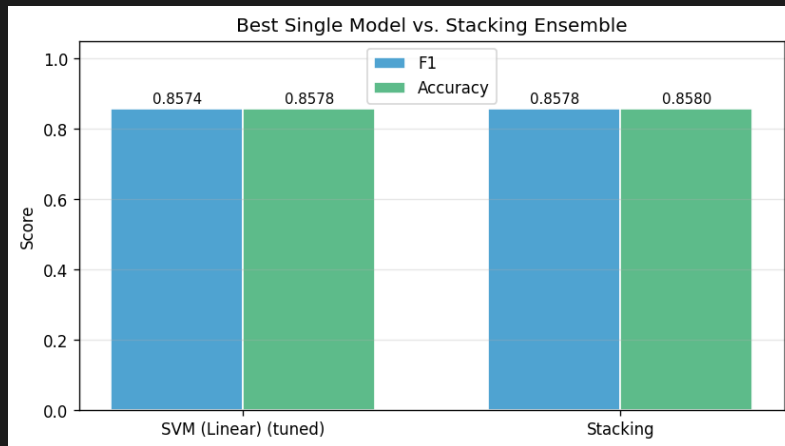
چهار پایه روی sparse: Hybrid، SVM، ComplementNB، LogisticRegression، RandomForest (n_estimators=300). stack_method=auto، cv=5، LogisticRegression: meta-learner. زمان آموزش حدود 683 ثانیه (11.4 دقیقه).

۲.۶. نتایج

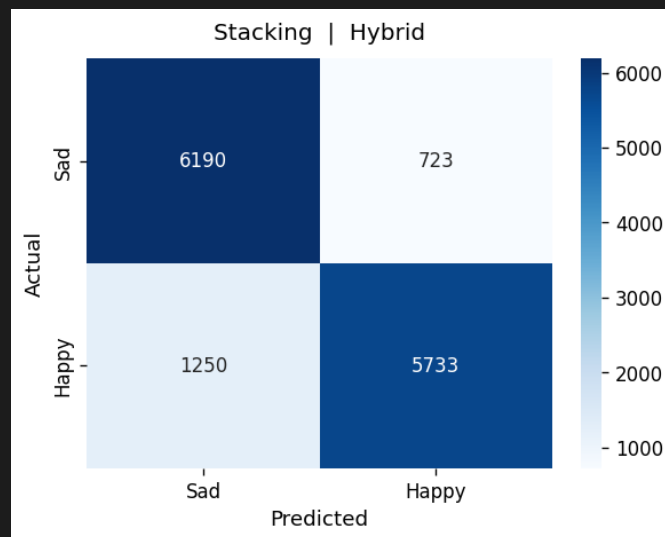
مدل	دقت	صحت	فراخوانی	F1
(Hybrid) Stacking	0.8580	0.8601	0.8580	0.8578
SVM (تکی) (tuned)	0.8578	0.8620	0.8578	0.8574

جدول ۸: Stacking در برابر بهترین تکی

بهبود F1: +0.0004 (0.04 نقطه درصد). کم ولی مثبت.



شکل ۸: مقایسه Stacking و بهترین تکی



شکل ۹: ماتریس درهم‌ریختگی Stacking + Hybrid

۳.۶. تحلیل

Stacking کمی بهتر از SVM tuned شد. بهبود کوچیکه چون پایه‌ها روی یه ماتریس مشترک آموزش دیدن و SVM خودش از قبل خیلی قوی بود. ولی meta-learner یاد گرفته کی به کدوم پایه اعتماد کنه.

۴.۶. نمونه‌های خطا و سقف دقت

متن	برچسب	پیش‌بینی
کیفیت خوب بود ولی نسبت به قیمت حجم غذا خوب بود	SAD	HAPPY
هسته زردآلو خیلی بد بود ولی برگه هلو خوب بود	SAD	HAPPY
آدرس اشتباه فرستاد ولی خود همبرگر خوب بود	SAD	HAPPY
از نظر مزه خوشمزه بود ولی حجم کم بود	SAD	HAPPY
سالاد سزار کاهو موند؛ سیب‌زمینی خوب بود	HAPPY	SAD
چیپس پنیر بدون سس تند سفارش دادم ولی تند فرستادید	HAPPY	SAD

جدول ۹: خطاهای واقعی Stacking

حدود 1,970 خطا از 13,896 تست (85.8% دقت). خیلی‌اشون نظر مختلط یا برچسب مشکوک‌ان، نه خطای تصادفی مدل. «کیفیت عالی. اما غذا سرد شده بود» (HAPPY) یا «خوشمزه بود ولی فوق العاده سرد» (SAD) نمونه‌ان که انسان هم مردد میشه. پس F1 حدود 0.86 هم موفقیت pipeline کلاسیکه هم یادآوری که داده 100% درست نیست.

۷. فاز ششم: ParsBERT (bonus)

بعد از اینکه Stacking Hybrid به $F1=0.8578$ رسید، سقف مدل‌های کلاسیک روی bag-of-words مشخص شد. فاز ۶ رفته سراغ transformer فارسی تا ببینیم context و fine-tuning چقدر می‌تونه از این سقف بالاتر بره. همه نتایج این بخش از نوت‌بوک 06_parsbert.ipynb روی Kaggle GPU اومده.

۱.۷. ParsBERT چیه و چرا اصلاً امتحانش کردیم؟

ParsBERT^۴ یه language model مبتنی بر Transformer هست که روی حجم زیادی متن فارسی pretrain شده. برخلاف TF-IDF/CountVectorizer که هر کلمه رو جدا می‌بینه، ParsBERT جمله رو دوطرفه می‌خونه: معنی «خوب» وقتی قبلش «نه» یا «ولی» باشه با حالت ساده فرق می‌کنه. برای نظرات Snapfood که پر از «خوب بود ولی...» هستن، همین تفاوت مهمه. مدلی که استفاده کردیم HooshvareLab/bert-base-parsbert-uncased هست - 12 لایه Transformer، tokenizer خودش (WordPiece)، توی پیاده‌سازی ما (src/bert_features.py) embed- ها dingling روی متن خام comment ساخته میشن، نه روی خروجی پیش‌پردازش فاز ۱. دلایل اینه که ParsBERT با فارسی طبیعی pretrain شده؛ مارک‌های EMO_/NEG_ و پاکسازی سنگین فاز ۱ برای BERT لزوماً کمک نمی‌کنه و گاهی گیجش می‌کنه.

۲.۷. پیاده‌سازی و محیط اجرا

نوت‌بوک روی Kaggle اجرا شد. های frozen feature از قبل کش شده بودن (X_train_bert.npy, X_test_bert.npy) - هر کدوم 768 بعد برای 55,583 train و 13,896 test. استخراج feature با ParsBertFeatureExtractor:

^۴BERT (Bidirectional Encoder Representations from Transformers): مدل زبانی Transformer دوطرفه. ParsBERT نسخه فارسی‌ش هست که روی متن فارسی pretrain شده.

pooling=mean, batch_size=32, max_length=128 روی hidden های state واقعی (نه فقط [CLS] خام)، چون برای extraction frozen معمولاً mean بهتر جواب میدهد.

سه مسیر جدا تست کردیم: **مسیر ۱ – ParsBERT Frozen + سرهای کلاسیک**. وزنهای Transformer ثابت موندن؛ فقط های classifier sklearn روی بردار 768-بعدی آموزش دیدن: SVM (Linear) کالیبره، LogisticRegression، HistGradientBoosting. بعد همون سه تا رو با StackingClassifier (LogisticRegression = meta-learner)، cv=5 ترکیب کردیم. آموزش stacking روی dense 768-بعدی حدود 3633 ثانیه (60 دقیقه) طول کشید.

مدل (frozen)	F1	دقت
SVM (Linear)	0.8445	0.8447
LogisticRegression	0.8438	0.8440
HistGradientBoosting	0.8357	0.8361
Stacking	0.8450	0.8452

جدول ۱۰: نتایج ParsBERT frozen + مدل های کلاسیک

هیچکدام از اینها از Stacking Hybrid فاز ۵ (0.8578) بالاتر نرفت. یعنی embedding از پیش آموزش دیده به تنهایی برای این دیتاست کافی نبود – حتی با stacking.

مسیر ۲ – end-to-end. Fine-tune اینجا کل ParsBERT روی برچسب های Snappfood fine-tune شد (num_labels=2, BertForSequenceClassification). train به 90% (50,024 نمونه) و 10% (5,559 نمونه) برای validation داخل train شکستیم (random_state=42). hyperparameterها: batch_size=32 epoch, 3, max_length=128, fp16=True, weight_decay=0.01, learning_rate=2e-5. بهترین checkpoint با load_best_model_at_end انتخاب شد.

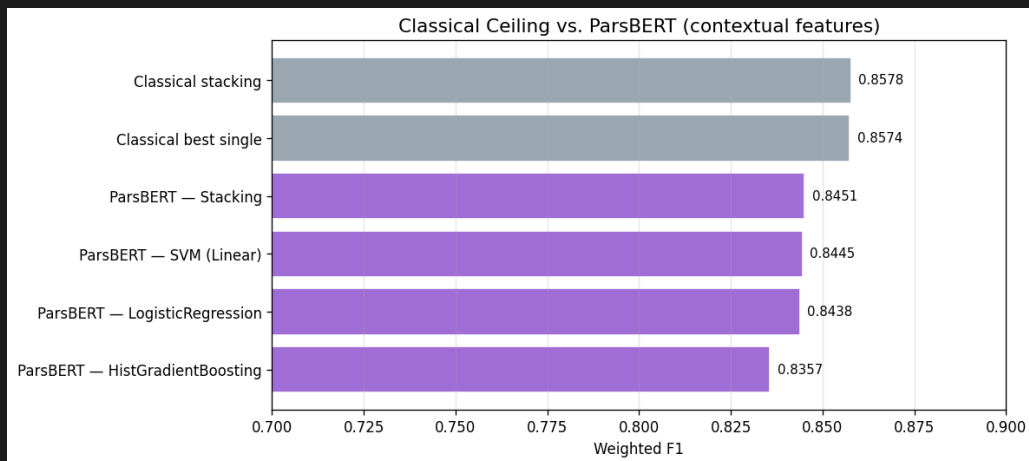
نتیجه روی test: F1=0.8730، دقت 0.8731 – یعنی +0.0152 نسبت به Stacking Hybrid. این اولین مدلی بود که مستقیم متن خام میخوند و از سقف کلاسیک عبور کرد.

مسیر ۳ – meta-ensemble. Extended پنج عضو قوی رو روی 10% blend از fit train کردیم و proba-bility خروجی شون رو به meta-learner (LogisticRegression، بهترین C=0.3) دادیم: SVM و LogReg روی Hybrid، HistGradientBoosting روی ParsBERT frozen، ComplementNB روی TF-IDF word، و fine-tuned ParsBERT. meta-learner روی 10% blend بعدی probability (5 عضو در 2 کلاس) آموزش دید. نتیجه test: F1=0.8713 – کمی پایین تر از fine-tune تنها (0.8730)، ولی هنوز خیلی بالاتر از pipeline کلاسیک. های meta-learner نشون میدن وزن اصلی روی ParsBERT fine-tuned افتاده؛ اعضای کلاسیک بعد از اضافه شدن transformer فاین تیون شده نقش کمتری دارن.

۳.۷. مقایسه با baseline کلاسیک

مرحله	F1	نسبت به Stacking Hybrid
(baseline) Stacking Hybrid	0.8578	0
ParsBERT frozen — stacking	0.8450	-0.0128
ParsBERT fine-tuned	0.8730	+0.0152
Extended ensemble	0.8713	+0.0135

جدول ۱۱: خلاصه bonus در برابر baseline



شکل ۱۰: مقایسه ParsBERT و pipeline کلاسیک

۴.۷. جمع‌بندی فاز ۶

خوب‌بیش چیه؟ fine-tune واقعاً سقف رو شکوند (1.5+ نقطه F1). بدیش چیه؟ نیاز به GPU، زمان آموزش بیشتر، وابستگی به transformers/HuggingFace. ParsBERT frozen حتی با stacking از Hybrid ضعیف‌تر بود — پس «داشتن BERT» به تنهایی کافی نیست؛ باید روی همین دامنه و همین برچسب‌ها fine-tune بشه. پس در نهایت: برای عبور از سقف bag-of-words، context لازمه؛ frozen کافی نیست، fine-tune لازمه. فاز ۶ bonus بود و deliverable اصلی پروژه همچنان فازهای ۱-۵ با مدل‌های کلاسیکه.

۸. فاز ۷۲ v2 (06__v2): end-to-end Snappfood-BERT

بعد از فاز ۶ رفتیم سراغ یه مسیر جدا توی نوت‌بوک 06__v2.ipynb — نه frozen feature، نه ParsBERT عمومی؛ اینجا مستقیم از HooshvareLab/bert-fa-base-uncased-sentiment-snappfood استفاده کردیم. انگار همون BERT فارسیه، ولی از قبل روی نظرات Snappfood برای sentiment آموزش دیده؛ یعنی از اول closer به همین دامنه‌ست.

۱.۸. چی کار کردیم؟

اول 70,000 ردیف اول CSV خام رو گرفتیم (نه split فاز ۱). Hazm، HAPPY/SAD normalize، 0/1، فیلتر طول با IQR، regex، dedup cleaning — 51 نظر تکراری حذف شد و 65,973 نمونه موند. اینجا split 30%/20%/50% با train/val/test random_state=42 بود: 32,986 train، 13,194 val، 19,793 test. پس از نظر protocol با فازهای ۳-۵ فرق داره — اونا 80/20 و preprocessing احساس‌محور فاز ۱ رو داشتن. روی checkpoint بالا CustomParsBERT ساختیم: BERT pooler → dropout=0.3 → Linear classifier. sifier، به علاوه L2=0.01 روی وزن‌های manual آموزش با PyTorch بود — AdamW، lr=1e-6، epoch. 6، batch=8، RTX 4070 Laptop هر epoch حدود 11 دقیقه طول کشید (جمعاً نزدیک 66 دقیقه training).

۲.۸. نتایج

بهترین accuracy validation توی epoch 3 بود: 0.9296. epoch 6 هم 0.9288 موند — یعنی overfit سنگین ندیدیم.

روی test (19,793 نمونه):

معیار	مقدار	توضیح
دقت (Accuracy)	0.9273	
صحت (Precision) weighted	0.9280	
فراخوانی (Recall) weighted	0.9273	
weighted F1	0.9274	بهترین خروجی این pipeline
F1 کلاس منفی	0.9272	9690 نمونه
F1 کلاس مثبت	0.9275	10103 نمونه

جدول ۱۲: نتایج test – 06_v2

حدود 1438 نظر (7.3%) اشتباه classify شدن – همون الگوی فازهای قبل: نظر مختلط «خوب بود ولی...» که به برجسب HAPPY/SAD برآش سخت.

خوبیش چیه؟ F1 رسید 0.9274 – خیلی بالاتر از stacking کلاسیک (0.8578) و حتی ParsBERT fine-tune عمومی (0.8730). بدیش چیه؟ split و preprocessing با pipeline اصلی یکی نیست، پس عدد 0.927 رو همیشه مستقیم با 0.858 یا 0.873 مقایسه کرد؛ ولی جهت improvement واضح: + checkpoint domain regularized. head + fine-tune end-to-end وزن‌ها توی outputs/v2_parsbert/final_model.pth ذخیره شد.

۹. مقایسه بهترین مدل هر فاز

اینجا به نگاه کلی به trajectory پروژه – از preprocessing تا domain-specific transformer فازهای ۱ و ۲ مدل classifier ندارن؛ پایه pipeline هستن.

فاز	نوت‌بوک	بهترین setup	F1
۱	01_preprocessing	پیش‌پردازش احساس‌محور	–
۲	02_vectorization	5 vectorizer (شامل Hybrid)	–
۳	03_base_models	SVM (Linear) tuned + Hybrid	0.8574
۴	04_vectorizer_selection	انتخاب Hybrid (همون SVM tuned)	0.8574
۵	05_stacking	Stacking + Hybrid	0.8578
۶	06_parsbert	ParsBERT fine-tuned (متن خام)	0.8730
۷۲	06_v2	Snappfood-BERT + CustomParsBERT	0.9274

جدول ۱۳: بهترین F1 هر فاز (جایی که classifier داشتیم)

حالا خوبیش چیه؟ trajectory واضح: مدل‌های کلاسیک روی bag-of-words سقفشون F1 0.86 بود. Stacking فقط +0.0004 داد نسبت به بهترین single – یعنی ensemble کلاسیک incremental بود، نه frozen ParsBERT breakthrough. حتی از Hybrid ضعیف‌تر بود (0.8450)؛ پس داشتن BERT به تنهایی کافی نیست. end-to-end fine-tune اولین جهش بود (0.8730، +0.0152 نسبت به (stacking v۲ با check-point مخصوص Snappfood رفت بالاتر (0.9274) – ولی یادت باشه split 50/20/30 و 70k subset داره، پس delta دقیق نیست.

بدیش چیه؟ protocol یکسان نیست بین فاز ۶ و ۷۲، preprocessing، split، حتی مدل پایه فرق می‌کنه. برای deliverable درسی، فازهای ۱-۵ + فاز ۶ (06_parsbert) روی همون split پروژه ترن؛ authoritative ۷۲ نشون میده اگه checkpoint domain داشته باشی و fine-tune end-to-end کنی، سقف عملی خیلی بالاتر میره.

پس در نهایت: pipeline classical کامل و reproducible روی split پروژه transformer؛ $F1=0.8578$ ؛ generic 0.8730؛ Snappfood-BERT ۷۲ 0.9274 روی protocol خودش — سه لایه answer بسته به اینکه چقدر resource و specificity domain داری.

۱۰. نتیجه‌گیری

یه pipeline کامل روی 69,479 نظر Snapfood: پیش‌پردازش احساس‌محور (نفی، emoji)، پنج vectorizer، 32+ آزمایش مدل، انتخاب Hybrid، Stacking با $F1=0.8578$. نکات کلیدی: URL قبل از Hazm؛ stopword احساسی حذف نشه؛ Hybrid برنده؛ SVM tuned sparse؛ قوی‌تر از درخت‌ها؛ Stacking بهبود کوچک ولی مثبت؛ خطاها اغلب نظر مختلط/برچسب نویزی؛ ParsBERT فاین‌تیون‌شده به 0.8730 رسید؛ مسیر ۷۲ (06_v2) با Snappfood-BERT و CustomParsBERT روی test به $F1=0.9274$ رسید (split 50/20/30، 70k subset). تحویل اجباری پروژه با فازهای ۱-۵ کامله؛ فاز ۶ bonus و بهبود واقعی با transformer فاین‌تیون‌شده بود؛ ۷۲ همون ایده رو با domain-specific checkpoint و loop training دستی ادامه داد.